

AEGIS SECURE CONVERGENCE SYSTEMS LLC

WORKING PAPER — v0.3

Ethereal Cryptography: Formal Definitions and Security Model

Introducing

***SPECTER** — Stateless Protocol for Ephemeral
Cryptographic Token Endorsement and Revocation*

Quantum Forward Secrecy
Ephemeral Assembly Security
Full Ethereal Security

Russell Aydelotte

Aegis Secure Convergence Systems LLC

Osceola, Indiana · May 2026

v0.3 · Subject to revision

© 2026 Russell Aydelotte, Aegis Secure Convergence Systems LLC. Licensed under CC BY-NC 4.0.

Abstract

This paper specifies and formally analyzes **Ethereal Cryptography**: an architectural approach to signing in which the complete private key is not held as a durable, persistent artifact, but assembled from threshold shares at the moment of use, used for a single operation, and erased within a bounded threat model, with epoch-scoped tokens. The idea that signing key material should not persist has a substantial lineage — forward secrecy, threshold and MPC signatures, key-insulated and intrusion-resilient schemes, and secure-erasure models — and **this work does not claim to originate it** (§10). Its contribution is narrower and threefold: (i) it treats the *key-existence window* Δt as an explicit, attested, audited operational parameter of a deployed system; (ii) it specifies a particular engineered composition — hardware-attested threshold assembly ceremonies, epoch-scoped tokens, secure erasure, and hybrid classical/post-quantum signing — not previously specified together, with a formal threat model and security games; and (iii) it defines a construction roadmap under which Δt reaches zero (full non-persistence) as standard-compatible threshold ML-DSA matures.

We define an **Ethereal Key Scheme (EKS)** and its security notions (Ephemeral Assembly Security, Epoch Isolation, and a conditional **Quantum Forward Secrecy** result). QFS is stated as a *named configuration* of forward secrecy holding under an explicit, empirical epoch-lifetime invariant — **not** a property stronger than the threshold/MPC state of the art, which already drives Δt to zero. Composed with a post-quantum signature (ML-DSA-87), the *target* end state, **Full Ethereal Security**, is that no complete signing key exists at any time.

We present **SPECTER** (Stateless Protocol for Ephemeral Cryptographic Token Endorsement and Revocation) as a deployed instantiation, analyze it against the EKS definition, and document precisely where the current build falls short of the full property (§9.4). This is a working specification with open problems, not a finished result.

Implementation status. This paper specifies the Ethereal Key Scheme and SPECTER as a *design* and proves the security properties of that design. The currently deployed SPECTER build conforms to this specification with three documented exceptions — fresh per-ceremony key generation (rather than deterministic epoch-keyed derivation), single-host simulation of distributed share custody, and a cryptographic threshold of $t = 2$ (with the 3-of-5 quorum enforced one layer up, at the governance voting gate). These are detailed in §9.4. As analyzed there, fresh-random key generation actually *strengthens* the EAS result — it makes the ephemeral key independent of the shares — while the specified deterministic derivation would require an additional secrecy condition on the epoch-binding input (§5.2, Case B); the single-host and threshold deviations do not affect the QFS, EAS, or Epoch Isolation results.

1. Introduction

Every known attack on a public-key cryptographic system — classical or quantum — depends on a single precondition:

The private key must exist in a recoverable form at some point in time.

Classical adversaries steal or factor their way to the private key. Quantum adversaries (Shor's algorithm) recover the private key from the public key in polynomial time — but only if the private key is *still useful* when the quantum computation completes.

The entire discipline of Post-Quantum Cryptography (PQC) (Bernstein & Lange 2017) addresses this by replacing mathematically-weak primitives (ECDLP, integer factoring) with mathematically-strong ones (lattice problems, hash-based signatures). This is necessary and correct. But it addresses the *algorithm* layer, not the *architecture* layer.

Ethereal Cryptography addresses the architecture layer.

If private key material does not persist — assembled from threshold shares only during an authorized ceremony, used once, and erased within a bounded threat model — then a quantum adversary arriving after the ceremony has no live key to steal, and any signature it later computes is *rejected by the protocol* because the epoch has expired. (An unbounded adversary can, of course, still *compute* a forgery from the public key; the guarantee is not that it cannot compute one, but that the protocol will not *accept* it — see §5.4.)

Key Insight

Traditional cryptography assumes that security depends on the computational hardness of recovering a persistent private key. Ethereum Cryptography rejects this assumption.

Instead, it enforces a stronger invariant:

A private key that does not persist cannot be stolen, and a key that no longer exists cannot be attacked.

We qualify this claim carefully: QFS is conditional on the engineering invariant that epoch lifetimes T_e are negligible compared to the quantum attack horizon $T_q(\lambda)$. This is an empirical deployment assumption, not a mathematical theorem. We state it explicitly wherever the result depends on it.

1.1 Our Contributions

These are stated relative to established work (§10), and are **not** claimed as originating the non-persistence idea. Several restate or engineer known notions: the QFS result (items 3–4) is a named configuration of forward secrecy under an empirical epoch invariant, already implicit in short-lived-key and quantum-authentication arguments; the EKS definition composes threshold sharing (Shamir 1979; Desmedt–Frankel 1989), key-insulated / intrusion-resilient models (Dodis et al. 2002; Itkis–Reyzin 2002), and secure erasure. What is new is the specific composition, its deployment (SPECTER), the treatment of the key-existence window as a measured parameter, and the roadmap to $\Delta t = 0$.

1. **Formal definition of an Ethereum Key Scheme (EKS)** — a tuple of algorithms satisfying Ephemeral Assembly, Single-Use Destruction, and Epoch Binding.
2. **A threat model for secure memory erasure** — explicitly bounding the adversary capabilities under which Destroy provides meaningful security, addressing the central load-bearing assumption.
3. **Formal definition of Quantum Forward Secrecy (QFS)** — a game-based security property capturing resistance to post-ceremony quantum attack, with a proof reducing to PRF security of HKDF and the UF-CMA property of the signing primitive.
4. **The Ephemeral-Quantum Gap Theorem** — a conditional theorem: if epoch lifetime T_e is negligible compared to $T_q(\lambda)$, then any EKS satisfying our threat model provides QFS.
5. **Full Ethereum Security (FES)** — the composed security property under a PQC signing algorithm.
6. **Liveness properties** — ceremony failure model and availability guarantees, complementing the safety properties.
7. **Multi-Ceremony Composition Theorem (Theorem 6.1)** — sequential-composition security proved for the deployed Case A and for Case B with the master secret hidden (a single PRF-oracle step plus UF-CMA); the all-shares Case B sub-case remains open (Open Problem 1). (*Upgraded from a conjecture in v0.3.*)
8. **SPECTER** — a concrete EKS construction with a deployed implementation, formally analyzed against the EKS definition.

1.2 Time as a Security Primitive

Traditional cryptography treats time as an external parameter. Keys exist indefinitely; adversaries race to break them before they are rotated. Security is a statement about how long a key *resists* attack.

Ethereal Cryptography internalizes time into the security model itself.

The validity of a key is not indefinite — it is explicitly bounded, and every protocol operation enforces that bound. Assemble produces a key valid for exactly one epoch. Sign consumes that key in one operation. Destroy closes the window. Verify rejects anything outside the window.

Security is therefore not a function of how long a key can resist attack. It is a function of whether the key remains valid long enough for an attack to matter. When epoch lifetimes are short relative to the quantum attack horizon, they cannot. The attack succeeds — and arrives to find an empty room.

This reframing has a precise consequence: the relevant security parameter is not the bit-length of the key but the ratio $T_e / T_q(\lambda)$. A 256-bit key with $T_e = 10$ years provides weaker temporal security than a 128-bit key with $T_e = 1$ hour, even if the 256-bit key is classically stronger. Ethereum Cryptography makes this trade-off explicit and exploitable by design.

1.3 Architectural vs. Mathematical Security

Two complementary approaches exist for resisting cryptographic attack:

Post-Quantum Cryptography replaces mathematically-weak primitives with mathematically-strong ones. It makes key recovery *harder* — requiring resources no known quantum computer possesses. Security depends on the intractability of the underlying hard problem.

Ethereal Cryptography aims to reduce the *value* of key recovery for already-expired epochs: an adversary who recovers an expired epoch's key gains nothing the protocol will accept, because the epoch is closed. This is forward secrecy for expired epochs, made conditional on an explicit temporal invariant ($T_e < T_q$) — **not** a claim that security is independent of computational hardness in general. Within a live epoch, and for the shares themselves, computational hardness still matters, and the same short-lived-key argument appears in the prior literature (§10).

Post-quantum cryptography strengthens the difficulty of key recovery. Ethereum Cryptography reduces the utility of expired-epoch key recovery, under an explicit temporal invariant. These approaches are orthogonal and composable.

Full Ethereum Security (§7) is achieved by composing both: the signing primitive resists quantum attack, and the architecture ensures that even a successful attack yields nothing useful.

2. Preliminaries

2.1 Notation

Symbol	Meaning
λ	Security parameter
PPT	Probabilistic polynomial-time
QPT	Quantum polynomial-time (BQP)
$\text{negl}(\lambda)$	Negligible function in λ
$\Pr[E]$	Probability of event E
A^O	Algorithm A with oracle access to O
$\leftarrow$	Uniform random assignment
$:=$	Definition
t_{now}	Current wall-clock time (distinct from threshold parameter t)
t	Threshold parameter in (t,n) -secret sharing
T_e	Epoch lifetime (wall-clock duration)
$T_q(\lambda)$	Quantum attack horizon for security parameter λ
T_e/T_q	Temporal security ratio — the key parameter of Ethereal Cryptography

Notation discipline: Throughout this paper, lowercase t denotes the threshold parameter in a (t,n) -secret sharing scheme. Wall-clock time is written t_{now} to avoid ambiguity. Epoch validity is written $\text{EpochValid}(\text{eid}, t_{\text{now}})$.

2.2 Threshold Secret Sharing

A **(t, n) -threshold secret sharing scheme** (Shamir 1979) is a tuple (Share, Reconstruct) such that:

- **Share**(s, t, n) $\rightarrow (\sigma_1, \dots, \sigma_n)$: splits secret s into n shares
- **Reconstruct**($\sigma_{\{i_1\}}, \dots, \sigma_{\{i_t\}}$) $\rightarrow s$: recovers s from any t shares
- **Privacy**: any subset of fewer than t shares is statistically independent of s

2.3 Epoch System

An **epoch system** is a tuple (EpochGen, EpochValid, EpochBind) where:

- **EpochGen**(λ) $\rightarrow (\text{eid}, T_{\text{start}}, T_{\text{end}})$: generates epoch identifier and validity window
- **EpochValid**($\text{eid}, t_{\text{now}}$) $\rightarrow \{0,1\}$: returns 1 iff current time $t_{\text{now}} \in [T_{\text{start}}, T_{\text{end}}]$
- **EpochBind**(eid, data) $\rightarrow \text{bytes}$: deterministically binds data to an epoch

Epochs are *non-overlapping* and *non-renewable*: once T_{end} passes, no new tokens can be issued for that epoch and no extension is possible.

Entropy requirement: The `aux` input to `EpochBind` MUST include fresh entropy or domain-separated context per epoch. Reuse of `aux` across distinct epochs without domain separation violates the Epoch Isolation invariant (Definition 5.3): if $\text{EpochBind}(\text{eid}_1, \text{aux}) = \text{EpochBind}(\text{eid}_2, \text{aux})$, then $\text{sk}_{\{e_1\}} = \text{sk}_{\{e_2\}}$, and a recovered key from one epoch directly applies to the other. Implementations MUST incorporate the epoch identifier as a mandatory component of `aux` (as SPECTER does via $\text{SHA3-256}(\text{eid} \parallel "||" \parallel \text{data})$).

Common Failure Modes in Epoch Binding

The following implementation errors collapse Epoch Isolation and MUST be treated as security-critical bugs:

Failure Mode	Consequence
<code>aux</code> reused across epochs without <code>eid</code>	$\text{EpochBind}(\text{eid}_1, \text{aux})$ may equal $\text{EpochBind}(\text{eid}_2, \text{aux})$; $\text{sk}_{\{e_1\}} = \text{sk}_{\{e_2\}}$ — epoch isolation collapses entirely
<code>eid</code> generated from low-entropy source	Epoch identifiers become predictable; adversary can anticipate <code>EpochBind</code> outputs
<code>eid</code> omitted from <code>EpochBind</code> salt	Two ceremonies with the same IKM produce identical keys regardless of <code>eid</code> value
<code>aux</code> generated before <code>t</code> shares are confirmed	Partial ceremony with <code>aux</code> fixed; abort+retry with same <code>aux</code> re-derives the same <code>sk_e</code> in a new epoch
<code>EpochBind</code> output truncated	Reduced domain separation; increased collision probability across epochs

Implementation rule: Generate `aux` fresh *inside* the ceremony window, after quorum confirmation, using a cryptographically secure random source. `eid` MUST be included as a mandatory prefix, not an optional component.

2.4 PRF Security

A function family $F = \{F_k : \{0,1\}^* \rightarrow \{0,1\}^\lambda\}$ is a **secure PRF** if no PPT distinguisher D can distinguish $F_k(\cdot)$ from a random oracle with non-negligible advantage. We model HKDF-SHA3-512 as a secure PRF in the random oracle model.

3. Threat Model and the Destroy Primitive

The Destroy operation is the load-bearing assumption of this paper. We address it explicitly rather than asserting it.

3.1 The Secure Erasure Problem

In practice, secure memory zeroing faces several implementation hazards:

- **Compiler optimization:** compilers may elide writes to memory that is about to go out of scope, treating zeroing as a dead write.
- **OS paging:** the kernel may swap pages containing key material to disk before Destroy is called.
- **CPU cache and register retention:** values may persist in L1/L2 cache lines or processor registers after memory is zeroed.
- **Language runtime semantics:** garbage-collected languages (Python, Java) do not guarantee when or whether objects are zeroed.
- **Hibernation and VM snapshots:** system state may be written to persistent storage at any point.

We do not require perfect physical erasure. We do not claim that key material is unrecoverable by every conceivable physical or microarchitectural means.

Instead, we require something more precisely bounded: within the Software Memory Adversary model, no adversary can recover *usable* key material after Destroy returns.

Destroy is therefore not a claim about physics. It is a protocol-enforced temporal boundary on key existence.

After Destroy, the key does not exist within the protocol's threat model — and protocol-level operations (Verify, Revoke, subsequent ceremonies) enforce that boundary. An adversary who circumvents Destroy via a physical attack is outside this model and requires hardware enclave mitigations (see §9.2).

Definition 3.1 — Software Memory Adversary (SMA)

A **Software Memory Adversary** is a PPT (or QPT) algorithm A that:

- Has access to the running process's observable outputs (tokens, public keys, witness records, network traffic)
- Can read process memory only *before* the ceremony begins or *after* Destroy returns — but NOT during the assembly/sign/destroy window
- Has no physical access to hardware (no cold-boot attacks, no DMA attacks, no power analysis)
- Cannot induce or observe paging of the key material region during the ceremony window
- Cannot interrupt or suspend the ceremony thread

Out of scope for the SMA model (acknowledged explicitly):

- Physical memory attacks (cold-boot, DMA, hardware probing)
- Hypervisor-level memory inspection
- OS kernel compromise
- CPU microarchitectural attacks (Spectre/Meltdown variants on key material)
- Hibernation/snapshot attacks during the ceremony window

Systems requiring stronger guarantees against physical adversaries should supplement EKS with hardware security enclaves (SGX, TrustZone) or a TPM-backed hardware root of trust.

Concrete mitigations within the SMA model: Implementations SHOULD apply the following OS/runtime controls to narrow the gap between the SMA model and physical reality:

- `mlock(2)` / `VirtualLock`: Pin key material pages to RAM, preventing the kernel from swapping them to disk during the ceremony window.
- `MADV_DONTDUMP` (Linux): Exclude key material regions from core dumps.
- `mlockall(MCL_CURRENT | MCL_FUTURE)`: Lock all current and future pages for the ceremony process.
- Real-time scheduling (`SCHED_FIFO` / `REALTIME_PRIORITY_CLASS`): Reduce preemption risk during the Sign→Destroy critical window.
- `madvise(MADV_WIPEONFORK)`: Prevent key material from being inherited by forked processes.

None of these eliminate physical memory attacks. They narrow the window of exposure within the SMA model and reduce implementation-layer risk. **The SMA model remains a protocol boundary, not a physics claim.** Enclave integration (§9.2) is the path to a stronger physical adversary model.

Limitation — secret-touching intermediates (stated openly). The SMA model permits an adversary to read process memory *after* Destroy returns. Destroy zeroizes the ephemeral key's SecureBuffer — but in a managed runtime (CPython) the *intermediate* values that touch secret material (the reconstructed Shamir secret *s*, HKDF

internal state in `hashlib`, and transient `bytes` objects) are heap-allocated and, when released, **freed but not overwritten**. An SMA adversary reading freed heap post-ceremony could recover them, and hence `s` (and, under deterministic derivation, `sk_e`). Satisfying the SMA model rigorously therefore requires either (i) confining *every* secret-touching intermediate to a zeroized `SecureBuffer` — difficult in CPython, and a motivation for a native-code assembly core — or (ii) explicitly narrowing Definition 3.1 to exclude freed-heap forensics, which is a real weakening of the model. The current build satisfies neither in full; this gap is part of Open Problem 5.

Definition 3.2 — Secure Destroy

Under the SMA model, **Destroy**(`sk_e`) satisfies *secure erasure* if:

1. All bytes of `sk_e` are overwritten with zeros using a memory barrier that prevents compiler reordering (e.g., `memset_s` in C, `explicit_bzero` in glibc/BSD (or `OPENSSL_cleanse` in OpenSSL); SPECTER's `SecureBuffer` zeroizes via `ctypes.memset`).
2. The zeroing occurs before the function returns to the caller.
3. The ceremony is executed atomically with respect to OS scheduling (no preemption between Sign and Destroy in the critical window).

Remark: Condition 3 is an implementation requirement, not a mathematical one. It can be enforced via OS scheduling primitives (e.g., `mlockall`, `MADV_DONTDUMP`, real-time scheduling priority). We treat it as a protocol requirement that implementations must satisfy, analogous to the requirement that a nonce is never reused in AES-GCM.

Remark on Python: CPython's `bytearray` and `ctypes` buffer types support explicit zeroing via `memset`. The standard `bytes` type does NOT guarantee zeroing. SPECTER's `SecureBuffer` uses `ctypes.create_string_buffer` with explicit `memset` calls to satisfy Definition 3.2 within the SMA model.

4. The Ethereal Key Scheme

Definition 4.1 — Ethereal Key Scheme (EKS)

An **Ethereal Key Scheme** is a tuple of algorithms:

$\text{EKS} = (\text{Setup}, \text{ShareGen}, \text{Assemble}, \text{Sign}, \text{Verify}, \text{Destroy}, \text{Revoke})$

satisfying the following:

Setup(λ) $\rightarrow$ `pp` Generates public parameters including epoch system, KDF parameters, and signing algorithm configuration.

ShareGen(`pp`, `t`, `n`) $\rightarrow$ (`pk`, σ_1 , ..., σ_n , `aux`) Generates `n` key shares for a (`t`,`n`)-threshold scheme. Returns a public key `pk` (to be published) and shares σ_i (distributed to share holders). The assembled private key `sk` is **never output**. `aux` contains entropy material for epoch binding.

Assemble($\sigma_{\{i\}}$, ..., $\sigma_{\{i_t\}}$, `eid`, `aux`) $\rightarrow$ (`sk_e`, `pk_e`, `W`) Given `t` shares and an epoch identifier `eid`, assembles:

- `sk_e`: ephemeral private key in a `SecureBuffer` (never written to disk)
- `pk_e`: ephemeral public key (published with tokens)
- `W`: witness record (quorum proof, not key material)

Protocol constraints:

- Assemble MUST verify $\text{EpochValid}(\text{eid}, t_{\text{now}}) = 1$ before proceeding; it MUST abort and call Destroy on any partial key material if the epoch is expired. This check is required for the EAS proof (§5.2).
- sk_e MUST be passed to Sign and then to Destroy within a single atomic ceremony window. It MUST NOT be serialized, logged, or returned to the caller.

Sign($\text{sk}_e, m, \text{eid}$) $\rightarrow \sigma_{\text{token}}$ Signs message m under sk_e , binding the signature to epoch eid . Called exactly once per assembled key.

Verify($\text{pk}_e, m, \sigma_{\text{token}}, \text{eid}, t_{\text{now}}$) $\rightarrow \{0,1\}$ Verifies signature validity, epoch validity at time t_{now} , and non-revocation of eid .

Destroy(sk_e) $\rightarrow \perp$ Securely zeros all bytes of sk_e per Definition 3.2. Called unconditionally after Sign (even on error — in a finally block).

Revoke($\text{eid}, \text{reason}$) $\rightarrow \text{rev_record}$ Permanently marks epoch eid as revoked.

Definition 4.2 — Correctness

An EKS is **correct** if for all $\lambda, \text{pp} \leftarrow \text{Setup}(\lambda)$, valid epochs eid (with $\text{EpochValid}(\text{eid}, t_{\text{now}}) = 1$), and messages m :

$$\Pr[\text{Verify}(\text{pk}_e, m, \text{Sign}(\text{sk}_e, m, \text{eid}), \text{eid}, t_{\text{now}}) = 1] = 1$$

where $(\text{pk}_e, \text{sk}_e, W) \leftarrow \text{Assemble}(t \text{ shares from ShareGen}(\text{pp}, t, n), \text{eid}, \text{aux})$.

4.1 Liveness Properties

A complete treatment requires both safety (bad things don't happen) and liveness (good things eventually happen). We define the following.

Definition 4.3 — Ceremony Liveness: An EKS satisfies *ceremony liveness* if: whenever at least t valid shares are presented and $\text{EpochValid}(\text{eid}, t_{\text{now}}) = 1$, Assemble completes and Sign produces a valid token within bounded time.

Definition 4.4 — Graceful Degradation: An EKS satisfies *graceful degradation* if: when fewer than t shares are presented, Assemble aborts cleanly — producing no partial key material, no partial token, and leaving no sk_e in memory. The epoch is not consumed (it may be retried within its validity window).

Definition 4.5 — Epoch Availability: An EKS satisfies *epoch availability* if: for any valid epoch eid with $T_{\text{end}} - T_{\text{start}} = T_e$, at least one ceremony can succeed within $[T_{\text{start}}, T_{\text{end}}]$ given a non-faulty quorum. (This is a protocol-level liveness property, not a cryptographic one — it depends on share holder availability.)

Remark on availability vs. security tradeoff: The threshold parameter t directly trades liveness for security. Lower t improves ceremony availability (fewer share holders needed) but weakens threshold privacy. A $t=3, n=5$ configuration tolerates two simultaneous share-holder failures while maintaining meaningful threshold security. (Note: SPECTER's deployed cryptographic threshold is $t=2$; the 3-of-5 quorum is enforced one layer up, at the LOCUST council *voting* gate that authorizes the ceremony — see §9.1 and §9.4.)

5. Security Properties

5.1 Standard Properties

Unforgeability (UF-CMA): No PPT adversary A , given pk_e and signing oracle access, can produce a valid (m^*, σ^*) for an unsigned m^* , except with probability $\text{negl}(\lambda)$. Reduces to UF-CMA of the underlying signature scheme.

Threshold Privacy: No adversary holding fewer than t shares learns anything about sk_e . Follows directly from Shamir (1979) statistical privacy.

5.2 Ephemeral Assembly Security (EAS)

Definition 5.1 — EAS Game:

```

Game EAS_A( $\lambda$ ):
  pp  $\leftarrow$  Setup( $\lambda$ )
  (pk,  $\sigma_1, \dots, \sigma_n$ , aux)  $\leftarrow$  ShareGen(pp, t, n)
  eid  $\leftarrow$  EpochGen( $\lambda$ )
  (sk_e, pk_e, W)  $\leftarrow$  Assemble({t shares}, eid, aux)
   $\sigma_{\text{token}} \leftarrow$  Sign(sk_e, m, eid)
  Destroy(sk_e)           // sk_e zeroed here

  // Adversary receives everything post-ceremony,
  // including ALL shares (worst case: all holders corrupt)
  (m',  $\sigma'$ )  $\leftarrow$  A(pp, pk, pk_e,  $\sigma_{\text{token}}$ , W, eid, m,  $\sigma_1, \dots, \sigma_n$ )

  // A wins by forging a signature on a fresh, never-signed message
  return 1 if m'  $\neq$  m and Verify(pk_e, m',  $\sigma'$ , eid, t_now) = 1

```

Definition 5.2 — EAS Security (under SMA model): An EKS satisfies EAS if for all PPT adversaries A operating within the Software Memory Adversary model:

$$\Pr[\text{EAS_A}(\lambda) = 1] \leq \text{negl}(\lambda)$$

Proof. The result depends on how the ephemeral key is bound to the shares, and the two cases diverge at exactly the point the EAS game stresses — the adversary is handed all n shares. We treat them separately.

Case A — independent ephemeral key (the deployed construction, §9.4 dev 1). Assemble draws sk_e from fresh randomness inside the ceremony, **independent of the reconstructed secret s** . Reconstructing the shares yields s , which is then statistically independent of sk_e — the shares reveal nothing about it. A 's only avenues are (i) reading sk_e from memory during the ceremony window, excluded by the SMA model, or (ii) forging (m', σ') from $(pk_e, \sigma_{\text{token}})$ with no signing oracle — a UF-CMA forgery under pk_e , succeeding with probability $\leq \text{negl}(\lambda)$. Both are negligible, so EAS holds **even against an adversary given all n shares**.

Case B — deterministic derivation (the specified target), $sk_e = \text{KDF}(s, \text{EpochBind}(\text{eid}, \text{aux}))$. Here sk_e is a deterministic function of $(s, \text{eid}, \text{aux})$. An adversary holding t shares reconstructs s offline and, if it also knows eid and aux , **recomputes sk_e directly — without invoking Assemble**, so the epoch-validity check inside Assemble provides no protection. Case B therefore does *not* satisfy EAS against an all-shares adversary unless one of:

- **(a)** the adversary is restricted to $\leq t-1$ shares, so s is hidden by Shamir privacy; or
- **(b)** aux contains a fresh, ceremony-scoped secret σ_{aux} that is never published (not in W , not on the wire) and is zeroized together with sk_e in Destroy — so that even an all-shares adversary lacks a KDF input. Under (b), σ_{aux} is itself key material and inherits Destroy semantics (and the §3 heap-intermediate limitation).

Absent (a) or (b), the deterministic variant is insecure while the epoch is live, and only *vacuously* secure post-expiry — Verify rejects on the epoch check, not on any cryptographic property. This is a genuine gap in the naive deterministic scheme, stated here rather than papered over.

Cross-epoch use (both cases). An adversary attempting to reuse recovered material for a different epoch e_2 gains nothing: $sk_{\{e_2\}}$ is computationally independent of $sk_{\{e_1\}}$ by Epoch Isolation (Definition 5.3, via Lemma 5.1).

The deployed build is Case A and satisfies EAS. Migrating to the specified deterministic derivation (Case B) is gated on adopting condition (b) — promoting σ_{aux} to destroyed key material — which is the corrected target for the §9.4 roadmap, not merely a backend-support item. ■

Lemma 5.1 — KDF Epoch Independence

Let $\text{KDF} = \text{HKDF-SHA3-512}$, modeled as a secure PRF. For any two distinct epoch identifiers $\text{eid}_1 \neq \text{eid}_2$ and any secret s :

$$\text{KDF}(s, \text{salt}=\text{EpochBind}(\text{eid}_1, \cdot)) \equiv_c \text{KDF}(s, \text{salt}=\text{EpochBind}(\text{eid}_2, \cdot))$$

(computationally indistinguishable)

Proof: $\text{EpochBind}(\text{eid}, \text{data}) = \text{SHA3-256}(\text{eid} \parallel "||" \parallel \text{data})$. For $\text{eid}_1 \neq \text{eid}_2$, $\text{EpochBind}(\text{eid}_1, \cdot) \neq \text{EpochBind}(\text{eid}_2, \cdot)$ with overwhelming probability (collision resistance of SHA3-256). HKDF with distinct salts and a PRF-secure hash function produces computationally independent outputs. This follows from the HKDF security analysis (Krawczyk 2010, Theorem 1). ■

5.3 Epoch Isolation (EI)

Definition 5.3: An EKS satisfies **Epoch Isolation** if for all PPT A and all distinct epochs $e_1 \neq e_2$:

$$\Pr[\text{Verify}(\text{pk}_{\{e_2\}}, m^*, A(\text{sk}_{\{e_1\}}, \text{pk}_{\{e_1\}}, \text{pk}_{\{e_2\}}), e_2, t_{\text{now}}) = 1] \leq \text{negl}(\lambda)$$

Proof: Follows directly from Lemma 5.1. Knowledge of $\text{sk}_{\{e_1\}} = \text{KDF}(s, \text{salt}_1)$ gives no information about $\text{sk}_{\{e_2\}} = \text{KDF}(s, \text{salt}_2)$ for $\text{salt}_1 \neq \text{salt}_2$, by PRF security of HKDF. Therefore A 's advantage in forging under $\text{pk}_{\{e_2\}}$ reduces to its UF-CMA advantage against the signing scheme with an unknown key — $\text{negl}(\lambda)$. ■

Remark (case split, matching §5.2). The proof above is the Case B (deterministic-derivation) treatment, where $\text{sk}_e = \text{KDF}(s, \text{salt})$. Under the deployed Case A (fresh-random ephemeral keys), $\text{sk}_{\{e_1\}}$ and $\text{sk}_{\{e_2\}}$ are drawn independently by construction, so Epoch Isolation holds *trivially* by key independence — there is no derivation linking them to reduce about. Both cases satisfy Epoch Isolation.

5.4 Quantum Forward Secrecy (QFS)

On the name. "Quantum Forward Secrecy" is an **exposition label**, not a new security notion. The underlying guarantee is short-epoch forward secrecy under the empirical invariant $T_e < T_q$, and it overlaps existing post-quantum forward-secrecy usage (cf. §10.2). We give it a game and a *conditional* theorem below to state the assumption and the guarantee precisely — not to dress a deployment assumption as a result. The substance is deliberately plain: *if a key's useful life is shorter than the time to break it, breaking it buys the adversary nothing*. §10 shows this reasoning is already present in the short-lived-key and quantum-authentication literature; our contribution here is to pin the invariant to a measured epoch bound (T_e) and a stated attack-horizon estimate (T_q), and to make the check enforceable at Verify.

Definition 5.4 — QFS Game:

```

Game QFS_A( $\lambda$ ):
  pp  $\leftarrow$  Setup( $\lambda$ )
  eid  $\leftarrow$  EpochGen( $\lambda$ ) with  $T_{\text{end}} = T_{\text{start}} + T_e$ 
  // Phase 1: Honest ceremony
  (sk_e, pk_e, W)  $\leftarrow$  Assemble(...)
   $\sigma_{\text{token}} \leftarrow$  Sign(sk_e, m, eid)
  Destroy(sk_e)           // sk_e zeroed; SMA model applies
  // Phase 2: Adversary is QPT; epoch has expired ( $T_e$  has elapsed)
  // A sees pk_e on the wire (it's in the token)
  // A attempts to forge a token for a new claim  $m^* \neq m$ :
  ( $m^*$ ,  $\sigma^*$ )  $\leftarrow$  AQPT(pk_e,  $\sigma_{\text{token}}$ , eid, W, pp)
  return Verify(pk_e,  $m^*$ ,  $\sigma^*$ , eid,  $t_{\text{now}}$ ) = 1
    // Note: epoch is expired, so EpochValid(eid,  $t_{\text{now}}$ ) = 0
    // Verify returns 0 regardless of signature validity
    // (the non-vacuous timed form, with A choosing  $t_{\text{sub}}$ , is in the remark below)

```

Definition 5.5 — Quantum Forward Secrecy: An EKS satisfies QFS if for all QPT adversaries A in the SMA model:

$$\Pr[\text{QFS_A}(\lambda) = 1] \leq \text{negl}(\lambda)$$

Fine-grained quantifier (important — corrects an asymptotic/wall-clock mismatch). For the classical-signature instantiation (Ed25519) this must be read as *concrete, time-bounded* security, **not** asymptotic QPT security. A full quantum polynomial-time adversary can run Shor and recover sk_e in polynomial time, so its asymptotic advantage against Ed25519 is *not* negligible — the statement " $\Pr \leq \text{negl}(\lambda)$ for all QPT A" would be **false**. The correct quantifier is over adversaries whose wall-clock runtime is bounded below $T_q(\lambda)$:

for all A running in time $< T_q(\lambda)$ in the SMA model, $\Pr[\text{QFS_A}(\lambda) = 1] \leq \text{negl}(\lambda)$.

The QFS guarantee for Ed25519 is therefore *fine-grained*, resting on the empirical invariant $T_e < T_q$. Under FES with ML-DSA-87 (§7), where no polynomial-time quantum key recovery is known, the quantifier reverts to standard (Q)PT and the T_q runtime bound is no longer needed.

Non-vacuity of the game. As written above, Phase 2 runs strictly *after* epoch expiry, so Verify returns 0 on the epoch predicate for *any* scheme that checks expiry — the game is unwinnable by construction, and its cryptographic content lives entirely in the live-epoch sub-case of Theorem 5.1. The substantive game lets A choose its own submission time t_{sub} , subject to runtime $< T_q(\lambda)$: A wins iff it outputs (m^* , σ^*) with $m^* \neq m$ and $\text{Verify}(pk_e, m^*, \sigma^*, eid, t_{\text{sub}}) = 1$. Under this formulation the theorem is non-trivial: no adversary can *both* recover sk_e *and* submit while the epoch is still live, because recovery costs $\geq T_q > T_e$. We take this timed game as the operative QFS definition; the post-expiry phrasing is retained only as its limiting case.

QFS does not prevent an adversary from eventually computing the private key.

Instead, it ensures that by the time such computation is possible, the key is no longer valid within the protocol.

Security is achieved not by resisting computation indefinitely, but by rendering delayed computation useless.

To be precise: QFS does not guarantee that a valid signature cannot be constructed after epoch expiry. It guarantees that no such signature can ever be accepted by the protocol.

Theorem 5.1 — The Ephemeral-Quantum Gap Theorem

Theorem: Let EKS be an Ethereum Key Scheme with epoch lifetime T_e , satisfying the SMA model (Definition 3.1) and Epoch Isolation (Definition 5.3). Let $T_q(\lambda)$ be the minimum time for any QPT algorithm to solve the hard problem underlying the signing primitive (e.g., ECDLP for Ed25519).

If $T_e < T_q(\lambda)$ is maintained as a deployment invariant, then EKS satisfies QFS.

Proof:

Let A be any adversary in the QFS game with wall-clock runtime bounded below T_q (the fine-grained quantifier of Definition 5.5; for ML-DSA-87 this bound is unnecessary and A may be any QPT algorithm). A wins by producing a valid (m^*, σ^*) such that $\text{Verify}(pk_e, m^*, \sigma^*, eid, t_{\text{now}}) = 1$.

Inspect the Verify predicate:

Verify returns 1 iff: (a) signature σ^* is valid under pk_e for message m^* , AND (b) $\text{EpochValid}(eid, t_{\text{now}}) = 1$, AND (c) eid is not revoked

Under the operative *timed* game (Definition 5.5, non-vacuity remark), A chooses its submission time t_{sub} subject to $\text{runtime} < T_q$. Branch on t_{sub} :

If $t_{\text{sub}} > T_{\text{end}}$ (A submits after the epoch has expired): condition (b), $\text{EpochValid}(eid, t_{\text{sub}}) = 1$, is FALSE, so $\text{Verify} = 0$ regardless of σ^* — A loses.

If $t_{\text{sub}} \leq T_{\text{end}}$ (A submits while the epoch is still live): winning requires a valid σ^* under pk_e , and the three sub-cases below show every path to such a σ^* is negligible — recovering sk_e is impossible within the window (recovery costs $\geq T_q > T_e$), forging without it is $\text{negl}(\lambda)$ by UF-CMA, and a cross-epoch forgery is $\text{negl}(\lambda)$ by Epoch Isolation.

Sub-case: A completes quantum attack before epoch expires ($T_{\text{attack}} < T_e$) A recovers sk_e via the quantum algorithm in time T_{attack} . By assumption $T_e < T_q(\lambda)$, and $T_{\text{attack}} \geq T_q(\lambda)$ for any QPT A (by definition of T_q as the minimum quantum attack time). Therefore $T_{\text{attack}} \geq T_q(\lambda) > T_e$ — contradiction: A cannot complete before T_e . This sub-case is impossible given the invariant.

Sub-case: A forges without recovering sk_e (using only $pk_e, \sigma_{\text{token}}$) A 's probability of producing a valid σ^* for $m^* \neq m$ is exactly A 's UF-CMA advantage against the signing scheme. Even for a QPT adversary against a classically-vulnerable scheme (Ed25519), this advantage requires either: (i) recovering sk_e (handled above), or (ii) a signing oracle (not available post-ceremony). Therefore this probability $\leq \text{negl}(\lambda)$ under standard UF-CMA assumptions.

Sub-case: A uses recovered $sk_{\{e\}}$ to forge for a different epoch e_2 By Epoch Isolation (Definition 5.3, proved via Lemma 5.1), A 's advantage reduces to the PRF advantage against HKDF-SHA3-512 — $\text{negl}(\lambda)$.

In all sub-cases, $\Pr[\text{QFS_A}(\lambda) = 1] \leq \text{negl}(\lambda)$. ■

Important qualification: The deployment invariant $T_e < T_q(\lambda)$ is empirical, not mathematical. For Ed25519, with T_q estimated on the order of years (extrapolated from the quantum-resource estimates of Roetteler et al. 2017 — not a direct wall-clock figure; see caveat below) and T_e on the order of hours, the gap is large. But T_q is an estimate with significant uncertainty — and that uncertainty cuts in EKS's favor.

Roetteler et al. give quantum *resource* counts (logical qubits, T-gates), not wall-clock time on future hardware. Converting resource counts to actual run time requires assumptions about:

- Fault-tolerant qubit quality and error correction overhead
- T-gate distillation throughput
- Qubit coherence time at the required circuit depth
- Classical control and measurement latency

Each of these factors inflates T_q beyond the raw circuit estimates. Current physical qubit error rates and coherence times suggest T_q is likely *longer* than Roetteler's optimistic estimates, providing a wider T_e/T_q gap in practice. System operators MUST nevertheless monitor T_q estimates and reduce T_e or migrate to ML-DSA-87 proactively. **QFS under Ed25519 is a time-bounded guarantee, not an unconditional one. Under FES with ML-DSA-87, this caveat becomes moot — T_q is effectively infinite for ML-DSA-87.**

5.5 Attested Key Existence (AKE)

The paper's distinctive contributions (§10.3) — governance-gating, epoch-scoping, and *audited existence* — are architectural claims until the last is given a formal object. We supply one here, deliberately lean, and **as an instantiation of existing accountability and transparency machinery, not a new security notion**. AKE is a property of the *witness chain and its attestation keys*, not of the signing ceremony: it formalizes that every key-existence window is attested and append-only auditable. It is the §10.1 trust-anchor displacement viewed from the other side — the persistent attestation/audit layer is precisely what makes key existence accountable.

Definition 5.6 — Attested Key Existence. Let the witness chain be an append-only, hash-chained log whose entries are signed under a persistent attestation key, and let Verify be extended with read access to that chain (beyond the Definition 4.1 interface — as the deployed SPECTER build's verification does, reading its append-only, Merkle-chained audit store). An EKS satisfies AKE if, for all PPT (or QPT) adversaries A , both hold except with probability $\text{negl}(\lambda)$: **(i) no unattested acceptance** — A cannot produce a token that Verify accepts without a matching, well-formed witness-chain entry recording the materialization of its key-existence window; and **(ii) record integrity** — A cannot forge, alter, reorder, or suppress a materialization record in the witness chain without detection.

The two clauses are jointly sufficient, and deliberately so. An adversary can of course run keygen offline on its own hardware, materializing a key while touching no log — but by clause (i) any token under such an unlogged key is rejected for lack of a matching attested record, so an offline key is harmless. Clause (ii) then need only guarantee the integrity of the records that *do* exist; it does not, and cannot, claim that every key materialization anywhere leaves a trace.

Soundness (informal, pending formal treatment — Open Problem 6). Both clauses reduce to standard transparency-log guarantees. (i) holds because Verify requires an attestation-signed witness entry, so acceptance without one demands forging the attestation key (unforgeability). (ii) holds because the log is hash-chained, so inserting, deleting, or reordering a record breaks a hash link (collision resistance of SHA3-256) and is detectable by any auditor holding a prior chain head. Detectability here assumes a *consistent chain-head view* across auditors: as in

CONIKS and CT, hash-chaining alone does not prevent a single log operator from presenting different heads to different parties (a split-view / equivocation attack); ruling that out requires gossip or multi-auditor cross-checking, which we fold into Open Problem 6. This is precisely the accountability/verifiability notion of Küsters, Truderung & Vogt (2010) instantiated for key-materialization events, realized by a CONIKS / Certificate-Transparency-style append-only log. We claim **no new notion** — we name the property the paper's distinctive rows already rely on, and defer a game-based definition and reduction to Open Problem 6.

6. Multi-Ceremony Composition

Sequential ceremonies are central to SPECTER's operational model. We prove sequential-composition security (Theorem 6.1) for the deployed Case A and for Case B with the master secret hidden, narrowing the former Open Problem 1 to the all-shares Case B sub-case.

Definition 6.1 — Sequential EKS Composition

A **sequential EKS execution** is a series of ceremonies $C_1, C_2, \dots, C_n$ over disjoint epochs $\text{eid}_1, \text{eid}_2, \dots, \text{eid}_n$ with $T_{\text{end}}(\text{eid}_i) < T_{\text{start}}(\text{eid}_{i+1})$. Each ceremony C_i uses the same underlying share set $(\sigma_1, \dots, \sigma_n)$ but a fresh epoch identifier.

Theorem 6.1 — Sequential Composition Security

Let EKS use HKDF-SHA3-512 (modeled as a PRF, §2.4) and a UF-CMA signature. Consider an adversary A observing the full public transcript of k sequential ceremonies over distinct epochs — $\{(\text{pk}_{\{e_i\}}, \sigma_{\{\text{token}, i\}}, W_i)\}_{i=1..k}$ — and attempting to forge a valid token for a fresh, never-signed epoch e_{k+1} . With the case structure of §5.2:

- **(Case A — deployed, fresh-random keys.)** Each $\text{sk}_{\{e_i\}}$ is drawn independently at random, so the transcript is statistically independent of $\text{sk}_{\{e_{k+1}\}}$; A 's forgery advantage equals its UF-CMA advantage, for *any* A (including one given all n shares).
- **(Case B — deterministic derivation, adversary holding $\leq t-1$ shares, so the master secret s stays hidden.)** $\text{Adv}^{\text{forge}}(A) \leq \text{Adv}^{\text{PRF_HKDF}}(\lambda) + \text{Adv}^{\text{UF-CMA}}(\lambda) = \text{negl}(\lambda)$.
- **(Case B, all-shares adversary.)** If A holds $\geq t$ shares it knows s , the PRF step below does not apply, and composition inherits the EAS Case B limitation (§5.2): it holds only under condition (b) — a fresh, destroyed ceremony secret σ_{aux} mixed into the derivation.

Domain Separation Invariant:

Each epoch derives its key material via a PRF with domain separation enforced by EpochBind. Therefore, even under full observation of all prior ceremonies — every public key, every token, every witness record — each epoch key is **computationally independent of all previous keys**. Observation of k ceremonies yields zero information about epoch $k+1$'s key material, by PRF security of HKDF-SHA3-512.

Proof (Case A). Immediate: independence of the $\text{sk}_{\{e_i\}}$ from $\text{sk}_{\{e_{k+1}\}}$ means the transcript can be simulated without $\text{sk}_{\{e_{k+1}\}}$, so any forger for e_{k+1} is a UF-CMA forger under an unqueried key. ■

Proof (Case B, s hidden). Model $\text{KDF}(\cdot) := \text{HKDF}(s, \cdot)$ as a function keyed by the secret s (the IKM) with the epoch-binding salt as its input (§2.4). (This keys the PRF on the IKM with the salt as input — the dual of Krawczyk's extractor-keyed position, where the salt is the extractor key and the PRK is the PRF key. Under §2.4's random-oracle modeling of SHA3, HMAC-SHA3 is a dual-PRF, so treating $\text{HKDF}(s, \cdot)$ as keyed by s is justified.) Compare two worlds:

- **Real:** each epoch answers $sk_{\{e_i\}} = \text{KDF}(\text{EpochBind}(eid_i, aux_i))$.
- **Ideal:** a truly random function RF replaces KDF; $sk_{\{e_i\}} = \text{RF}(\text{EpochBind}(eid_i, aux_i))$.

Because the salts $\text{EpochBind}(eid_i, \cdot)$ are distinct across epochs (collision resistance of SHA3-256, Lemma 5.1), RF returns $k+1$ *independent uniform* keys, so in the Ideal world $sk_{\{e_{k+1}\}}$ is independent of everything A observes.

Bounding the gap. A distinguisher D simulates the whole game — for each epoch i it queries its oracle F at $salt_i$ to get $sk_{\{e_i\}}$, computes $pk_{\{e_i\}}$, $\sigma_{\{\text{token}, i\}} = \text{Sign}(sk_{\{e_i\}}, m_i)$, and W_i , runs A on the transcript, and outputs 1 iff A's forgery verifies. If $F = \text{KDF}$ this is Real; if $F = \text{RF}$, Ideal. Hence

$|\Pr[A \text{ wins} \mid \text{Real}] - \Pr[A \text{ wins} \mid \text{Ideal}]| \leq \text{Adv}^{\text{PRF_HKDF}}(D)$. D never needs s — only oracle access — which is exactly why the entire transcript (all k keys at once) is handled in a *single* step, with no per-epoch hybrid.

Ideal world. D publishes the target epoch's public key by querying its oracle at $salt_{\{k+1\}}$, running the signature keygen on the resulting seed, and giving $pk_{\{e_{k+1}\}}$ to A. Here $sk_{\{e_{k+1}\}} = \text{RF}(salt_{\{k+1\}})$ is a uniform seed, independent of the transcript (which depends only on RF at salts $1..k$), and A is given **zero** signing queries under $sk_{\{e_{k+1}\}}$ — so a valid token for e_{k+1} is exactly a (key-only) UF-CMA forgery: $\Pr[A \text{ wins} \mid \text{Ideal}] \leq \text{Adv}^{\text{UF-CMA}}(\lambda)$. This step assumes the signature scheme's keygen consumes a uniform seed and that the UF-CMA challenger's key distribution equals $\text{keygen}(\text{uniform})$ — which holds for Ed25519 and for seeded ML-DSA keygen, and is the same seam as §9.4 deviation 1 (deterministic seeded keygen).

Combining — and carrying the salt-distinctness term from Lemma 5.1 (collision resistance of SHA3-256) so the accounting is honest: $\text{Adv}^{\text{forge}}(A) = \Pr[A \text{ wins} \mid \text{Real}] \leq \text{Adv}^{\text{PRF_HKDF}}(\lambda) + \text{Adv}^{\text{UF-CMA}}(\lambda) + \text{Adv}^{\text{coll_SHA3}}(\lambda) = \text{negl}(\lambda)$. ■

Remark (relation to the k-ceremony hybrid). The intuition "peel off one observed ceremony at a time, each hop a PRF step" is realized here in a *single* step, because $\text{HKDF}(s, \cdot)$ with varying salt is a PRF *oracle*: replacing it wholesale with a random function makes all epoch keys independent at once. A literal k -hop hybrid gives the looser bound $k \cdot \text{Adv}^{\text{PRF}} + \text{Adv}^{\text{UF-CMA}}$ and would require the distinguisher to hold s (or an oracle) to fill the un-swapped epochs; the single-oracle form above is both tighter and avoids that. This upgrades the former Conjecture 6.1 to a theorem for Case A and Case B (s hidden); the all-shares Case B case remains gated on the σ_{aux} condition (§5.2, condition (b)). (*Newly added in v0.3; pending external review.*)

7. Full Ethereal Security

Definition 7.1 — Full Ethereal Security (FES)

An EKS achieves **Full Ethereal Security** if:

1. The underlying signature scheme is **QPT-unforgeable** (e.g., ML-DSA-87) — no known quantum attack applies.
2. The EKS satisfies **QFS** (Definition 5.5).
3. The EKS satisfies **EAS** under the SMA model (Definition 5.2).
4. The EKS satisfies **Epoch Isolation** (Definition 5.3).

Theorem 7.1: Under FES, no QPT adversary (within the SMA model) can forge a valid token for any epoch, past or future, with probability better than $\text{negl}(\lambda)$ — for Case A and for Case B with the master secret hidden; the all-shares Case B sub-case is gated on the σ_{aux} condition (§5.2(b)).

Proof:

- *Future epochs*: QPT UF-CMA of ML-DSA-87 (the appropriate notion of unforgeability against quantum adversaries; see Boneh & Zhandry 2013) prevents forgery without sk_e . ML-DSA-87 security reduces to the hardness of the Module Learning With Errors (MLWE) problem, for which no QPT algorithm is known (NIST FIPS 204).
- *Current/past epoch forgery post-ceremony*: EAS (Definition 5.2) ensures sk_e is unrecoverable after Destroy under the SMA model.
- *Past epoch forgery using recovered share material*: Epoch Isolation (Definition 5.3) via Lemma 5.1 ensures $\text{negl}(\lambda)$ advantage.
- *Cross-epoch forgery from sequential observations*: bounded to $\text{negl}(\lambda)$ by **Theorem 6.1** (sequential composition) for Case A and for Case B with the master secret hidden; the all-shares Case B sub-case is gated on the σ_{aux} condition (§5.2(b)). ■

Remark: Under FES with a QPT-hard signing primitive, the QFS deployment invariant $T_e < T_q(\lambda)$ is no longer necessary for security — ML-DSA-87 provides unconditional resistance to Shor's algorithm, so even if a quantum computer completes in $T_{\text{attack}} < T_e$, it cannot forge under the signing scheme. FES is therefore independent of the T_e/T_q gap.

8. HNDL and the Signature Scheme Distinction

Clarification: Harvest-Now-Decrypt-Later (HNDL) is primarily a threat against *encryption* schemes: an adversary captures ciphertext now and decrypts when quantum hardware is available. For *signature* schemes, there is no ciphertext to decrypt — so the standard HNDL framing is inapplicable.

The relevant threat for signature schemes is subtly different and more interesting:

Token Harvest for Future Forgery: An adversary harvests signed tokens from the wire. The tokens contain pk_e (the Ghost Key public key). When a quantum computer is available, the adversary runs Shor's ECDLP on pk_e to recover sk_e , then uses sk_e to forge new tokens for any identity and scope.

This attack requires:

1. A valid, unexpired epoch at the time of forgery (to pass Verify condition (b))
2. sk_e recovered before the epoch expires

EKS resistance: The epoch expiry check in Verify provides complete resistance: by the time quantum hardware could recover sk_e (T_q on the order of years; see §5.4), the epoch has long since expired. The harvested tokens cannot be used as forgery vehicles because their epoch is revoked.

Under FES (ML-DSA-87): The attack is impossible regardless of epoch timing, because Shor's ECDLP does not apply to ML-DSA-87.

9. SPECTER as an EKS Instantiation

SPECTER (Stateless Protocol for Ephemeral Cryptographic Token Endorsement and Revocation) is the first formally specified instantiation of an EKS.

EKS Component	SPECTER Implementation
Setup	PQCAadapter init, HKDF-SHA3-512 parameters
ShareGen	Shamir (t,n) over Ghost Key seed; shares on USB HRoT
Assemble	Ghost Key ceremony (quorum-gated): Shamir reconstruct → HKDF → ephemeral keypair in SecureBuffer (see §9.4)
Sign	Ed25519 (fallback) or ML-DSA-87 (PQC mode)
Verify	SPECTER token verification: dual-sig check (Ed25519 × ML-DSA) + epoch validity + revocation list
Destroy	SecureBuffer.destroy() — mlock'd buffer, ctypes memset, destructor fallback
Revoke	Epoch revocation in SPECTER audit store (Merkle-chained)

9.1 Liveness in SPECTER

SPECTER satisfies ceremony liveness (Definition 4.3): the deployed LAIRS integration runs SPECTER at its minimum cryptographic threshold $t=2$, with the 3-of-5 quorum enforced one layer above, at the LOCUST council *voting* gate that authorizes the ceremony. A $t=3$, $n=5$ ceremony configuration tolerates two simultaneous share-holder failures (see §9.4 for the deployed-vs-specified threshold reconciliation).

SPECTER satisfies graceful degradation (Definition 4.4): Assemble raises `CeremonyAbortError` on insufficient shares; the SecureBuffer is zeroed in the exception handler before propagation.

Epoch availability (Definition 4.5) depends on council member availability — a protocol liveness concern outside the cryptographic model. The LOCUST dashboard provides real-time availability monitoring.

9.2 Destroy in SPECTER

SPECTER's SecureBuffer (specter/crypto/memory.py) allocates a mutable bytearray, pins it in RAM with `mlock(2)` so it cannot be swapped to disk during the ceremony, and zeroizes it with a single compiler-resistant `ctypes.memset` on destroy, with a destructor fallback:

```
import ctypes, ctypes.util
_libc = ctypes.CDLL(ctypes.util.find_library("c"), use_errno=True)
class SecureBuffer:
    def __init__(self, size: int):
        self._size = size
        self._buf = bytearray(size)          # mutable; not the immutable `bytes`
        addr = ctypes.addressof((ctypes.c_char * size).from_buffer(self._buf))
        self._locked = (_libc.mlock(ctypes.c_void_p(addr),
                                    ctypes.c_size_t(size)) == 0)
        if not self._locked:
            # RLIMIT_MEMLOCK / no privilege
            raise CeremonyAbortError(
                "mlock failed – refusing to hold key material in swappable memory")
        self._destroyed = False
    def destroy(self):
        if self._destroyed:
            return
        addr = ctypes.addressof(
            (ctypes.c_char * self._size).from_buffer(self._buf))
        ctypes.memset(addr, 0, self._size)    # zeroize (not elided by CPython)
        if self._locked:
            _libc.munlock(ctypes.c_void_p(addr), ctypes.c_size_t(self._size))
        self._destroyed = True
    def __del__(self):
        self.destroy()                       # destructor fallback
```

This satisfies Definition 3.2 within the SMA model. It does NOT protect against OS paging or physical memory attacks — those require hardware enclave support (a TPM-backed hardware root of trust, planned in the LAIRS roadmap).

Specification requirement (mlock must be fatal on failure). `mlock(2)` is subject to `RLIMIT_MEMLOCK` and can fail silently without `CAP_IPC_LOCK` or privilege. Recording the lock result and then *proceeding unlocked* — as an earlier build did — is a specification bug: unlocked key pages may be swapped to disk, violating the SMA model. The specification **REQUIRES** that a failed lock **aborts the ceremony before any secret is written** (as the code above now does). Operators **MUST** provision `RLIMIT_MEMLOCK` (or grant `CAP_IPC_LOCK`) so that ceremonies fail closed on a genuine lock failure rather than continuing in swappable memory.

9.3 SPECTER Security Status

Property	Ed25519 fallback	ML-DSA-87 (PQC)
Classical UF-CMA	YES	YES
QPT UF-CMA	NO (Shor's)	YES (MLWE hardness)
EAS (SMA model)	YES	YES
Epoch Isolation	YES	YES
QFS ($T_e < T_q$ gap)	YES — conditional	YES — unconditional
Sequential composition	Theorem 6.1	Theorem 6.1
Full Ethereum Security	NO	YES

The two columns above are the security endpoints; the deployed backend selects the strongest available at ceremony time, in order: **ML-DSA-87** (liboqs shim — the current live backend) → **ML-DSA-65 / Dilithium-3** (intermediate, via the oqs package) → **Ed25519** (classical dev fallback). Only the ML-DSA-87 path yields Full Ethereum Security.

9.4 Implementation Status and Known Deviations

The formal model (§§3–8) specifies SPECTER's intended design. The deployed build matches it except in the three respects below, which we document explicitly for reproducibility. The threshold and single-host deviations do not affect the QFS, EAS, or Epoch Isolation results; the key-generation deviation (1) *strengthens* EAS, as analyzed below.

1. **Key generation.** §5.2 specifies the ephemeral key as a deterministic derivation, $sk_e = \text{KDF}(s, \text{salt} = \text{EpochBind}(\text{eid}))$. The current build instead generates a *fresh* ephemeral keypair inside the ceremony (in the mlock'd SecureBuffer) and destroys it after a single signature; the HKDF-derived seed is computed but not yet threaded into key generation. This is because liboqs 0.15.0 exposes deterministic seeded keygen for KEMs (`OQS_KEM_*_keypair_derand`) but not for ML-DSA signatures. Its security effect is the opposite of a weakening: under fresh-random keys the ephemeral key is *independent of the shares*, so even an adversary holding all n shares learns nothing about it — this is exactly **Case A** of the EAS proof (§5.2), and it is what makes EAS hold in the deployed build. The specified deterministic derivation (**Case B**) is *not* merely pending backend support: as §5.2 shows, binding sk_e to the reconstructed secret makes it recomputable by an all-shares adversary unless the epoch-binding input `aux` includes a fresh, ceremony-scoped secret that is destroyed with sk_e (condition (b)). The corrected migration target is therefore *deterministic derivation with a destroyed secret aux component*, gated on both that design change and SIG-level seeded-keygen support in the PQC backend.
2. **Quorum distribution.** §2.2 and §9.1 describe an (t,n) ceremony in which each principal independently holds a share. The deployed single-host build generates the master secret on the ceremony host and splits/reconstructs it locally, while principals (Hardware Identity Anchors) contribute attestations and an optional ML-KEM seed contribution. Fully distributed share custody — each principal holding its own share — is the intended design; the deployed single-host build *simulates* distribution.
3. **Threshold.** The deployed SPECTER cryptographic threshold is $t=2$ (its minimum). The 3-of-5 governance quorum is enforced at the LOCUST council voting layer that authorizes each ceremony, not inside the SPECTER threshold.

A consequence worth stating plainly (deviations 1 + 2 together). Because the ephemeral key is fresh-random (dev 1) *and* the HKDF-derived seed is not yet threaded into key generation, in the current deployed build the Shamir

reconstruction performs **no cryptographic work on the signing key** — the reconstructed secret s does not determine sk_e . The shares therefore gate the ceremony only *governance-wise* (quorum authorization through the council / HIA layer), not cryptographically. This is a real limitation, not a hidden strength: cryptographic binding of the signing key to a t-of-n secret is precisely what the Case B migration (§5.2, with a destroyed σ_{aux}) is meant to establish. Until then, threshold *privacy of the signing key* is provided by non-persistence and fresh randomness, not by Shamir sharing of that key. A reviewer will observe this; we state it first.

10. Relation to Existing Work

10.1 Forward Secrecy (TLS)

TLS forward secrecy protects session keys: compromise of the long-term server key does not reveal past session keys. However, the long-term server key *persists* and is a permanent attack target. EKS eliminates the long-term asymmetric key entirely. Where TLS provides forward secrecy *despite* a persistent long-term key, EKS provides it *because* no persistent key exists.

Precise distinction: TLS with DHE/ECDHE generates ephemeral session keys per handshake, but the *identity* key (server certificate private key) is long-lived. An adversary who compromises the identity key can impersonate the server in future connections. EKS has no long-lived *signing* key — only epoch-scoped assembled keys that are destroyed immediately.

Displacement, not elimination (important caveat). EKS removes the long-lived signing key, but Verify must still authenticate that pk_e belongs to a legitimate ceremony — and it does so via the witness record W , the hash-chained witness log, and the revocation registry, all of which are anchored by *persistent* keys (TPM / HRoT attestation keys, audit-store signing keys). The persistent root of trust is therefore **displaced** from the signing key to the attestation and audit layer, not eliminated. This is deliberate and defensible — those persistent keys are non-exportable, hardware-bound, and used only to attest and anchor, never to sign tokens — but the honest claim is *displacement of the trust anchor*, not "no persistent key exists anywhere."

10.2 Signal Protocol / PQXDH

Signal's PQXDH (Post-Quantum Extended Diffie-Hellman, 2023) achieves post-quantum forward secrecy for messaging via ephemeral key deletion after each key exchange. This is conceptually close to EKS in motivation.

Precise distinctions:

- PQXDH targets *encryption* key exchange (KEM); EKS targets *signing* and authorization tokens. The threat models are different.
- PQXDH ephemerals are pre-generated and stored (prekey bundles on a server) until consumed. EKS keys are never stored at all — assembled only at the moment of ceremony.
- PQXDH has no threshold assembly or quorum requirement; the ephemeral key is under sole custody of one device. EKS requires t-of-n quorum, making key assembly a governed, auditable event.
- PQXDH's "deleted ephemeral" provides forward secrecy contingent on secure deletion from device storage. EKS's key is never in storage — it exists only in RAM for the duration of the ceremony window.

EKS and PQXDH are complementary: PQXDH addresses the communication layer; EKS addresses the authorization token layer.

10.3 Threshold and MPC Signatures (FROST, GG20/CGGMP, threshold ML-DSA)

Threshold and MPC signature schemes let n parties sign without any single party holding the full key. With distributed key generation (DKG), the complete key **never exists at any party at any time**, including at generation. FROST (Komlo–Goldberg 2020; RFC 9591, CFRG 2024) provides this for Schnorr/Ed25519; the GG18/GG20 and CGGMP21 lines provide it for ECDSA and underpin a commercial MPC-custody industry (Fireblocks, Coinbase, Zengo). For these schemes, "the key does not exist" is literal, not a metaphor.

Honest positioning relative to EKS. These schemes already achieve the endpoint EKS aims toward — key non-existence, i.e. $\Delta t = 0$. SPECTER v1's assemble-sign-destroy lifecycle, by contrast, momentarily materializes the complete key in a bounded, attested window (§9.4, Gap 1), and uses a dealer at genesis. **EKS is therefore not "stronger" than threshold signing on the key-existence axis; on that axis it is currently weaker.** What EKS adds is orthogonal: hardware-attested, governance-gated, epoch-scoped ceremonies with an audit trail that treats key existence itself as the logged event. The correct long-term relationship is *adoption, not competition*: the construction roadmap migrates the assembled-key lifecycle onto threshold signing directly — FROST for the classical path, and standard-compatible threshold ML-DSA (Celi et al., ePrint 2026/013; Quorus, USENIX Security 2026) for the post-quantum path — at which point Δt reaches zero and the distinction collapses. The residual threat surface today is the *shares* (mitigated by hardware custody) **and** the brief assembled-key window; the latter closes only under threshold signing.

Cautionary note — the threshold-ML-DSA frontier is young and unsettled. We deliberately restrict the roadmap's post-quantum candidates to Celi et al. and Quorus. An earlier one-round candidate, TALUS (Kao & Chang, arXiv:2603.22109), was broken by a passive polynomial-time key-recovery attack (Niot, IACR ePrint 2026/1386): its removed s_2 rejection check leaks a linear equation per signature (full recovery after $\approx 2^{30}$ signatures under one key), and its MPC variant's Feldman commitments are invertible by Gaussian elimination with no lattice problem to solve. The authors' v4 revision (arXiv:2603.22109v4, 23 Jul 2026) rewrites the security analysis to address the attack, but that fix is not yet externally vetted. TALUS is therefore **not** treated as a viable target here. This is itself an instance of the Vanish/Unvanish lesson (§10.6): non-persistence and threshold constructions concentrate attacks exactly where a claimed hard problem turns out to be bypassable — which is why the roadmap gates the post-quantum phase on external cryptanalysis, not on a scheme's own security claims.

10.4 One-Time Signatures (Lamport, SPHINCS+)

One-time signature schemes restrict key use to prevent cryptanalytic attack. EKS is orthogonal: the key is destroyed after one use not to prevent cryptanalysis but to prevent theft. The security arguments are independent and composable. A one-time signature scheme instantiated within EKS would provide both properties simultaneously.

10.5 Hardware Security Modules (HSMs)

HSMs protect persistent keys via tamper-resistant hardware. EKS provides a complementary guarantee: the assembled key does not persistently exist *anywhere*, including HSM memory. The shares (analogous to HSM key material) are protected elements, but they are transformed into an ephemeral key that neither lives in the HSM nor anywhere else. EKS + HSM (for share storage) is a natural composition and is realized in SPECTER via USB HRoT share custody.

10.6 Key-Insulated / Intrusion-Resilient Models and Secure Erasure

A parallel academic line formalized *bounded key existence and exposure* — the space EKS operates in — and must be cited as ancestry, not empty ground. Key-insulated public-key cryptography (Dodis–Katz–Xu–Yung 2002) and intrusion-resilient signatures (Itkis–Reyzin 2002) contain the damage of per-epoch key compromise; proactive secret sharing (Herzberg–Jarecki–Krawczyk–Yung 1995) refreshes shares so an adversary must compromise a threshold within a single epoch. On the erasure side, revocable backup (Boneh–Lipton 1996), "how to forget a secret" (Di Crescenzo–Ferguson–Impagliazzo–Jakobsson 1999), and the Ephemerizer (Perlman 2005) formalize assured deletion; Vanish (Geambasu et al. 2009) and its break (Unvanish) establish that any non-persistence claim carries a specific burden — the temporal and spatial windows in which material exists must be stated explicitly, because attacks concentrate exactly there.

Positioning. EKS does not introduce bounded key existence as a notion; those models predate it. What EKS adds is to make the existence window an explicit, attested, audited operational parameter and to state that window precisely (§3 threat model, §9.4 deviations) — meeting the burden the Vanish/Unvanish episode established.

Summary of distinctions: The following table characterizes precisely how EKS differs from related schemes along the dimensions that matter most:

Property	TLS-FS	PQXDH	FROST/MPC	HSM	EKS (SPECTER v1)
Full key ever exists	YES	YES	NO (with DKG)	YES	Target: no; v1: yes, briefly + dealer genesis (§9.4)
Assembled key stored	N/A	YES (prekey)	N/A (never assembled)	YES	NO
Quorum required for signing	NO	NO	YES	NO	YES
Epoch-scoped validity	NO	NO	NO	NO	YES
Governance-gated + audited existence	NO	NO	NO	Partial	YES
Post-quantum sign primitive	Partial	YES	Emerging (threshold ML-DSA)	Varies	YES (PQC mode)
Post-ceremony key unrecoverable	NO	Partial	YES (never exists)	NO	YES (SMA model, per bounded window)

The FROST/MPC column already achieves the strongest key-existence posture (the key never exists); EKS's distinguishing rows are governance-gating, epoch-scoping, and audited existence (the last formalized as Attested Key Existence, §5.5) — not key-existence, where v1 is currently weaker.

11. Open Problems

1. **All-shares Case B sequential composition** — Theorem 6.1 now proves sequential composition for Case A and for Case B with the master secret hidden. The remaining open case is Case B against an adversary holding $\geq t$ shares (s known), which requires the destroyed- σ_{aux} construction of §5.2(b); a game-based proof for that construction is future work.
2. **QROM security** — Extend QFS proof to the quantum random oracle model, addressing the use of SHA3-512 in HKDF.
3. **Formal EAS reduction under physical adversary** — Extend the SMA model to include hardware enclave guarantees (SGX, TrustZone) and provide EAS security against physical memory adversaries. A TPM-backed hardware-root-of-trust integration in LAIRS is the intended realization.
4. **Share compromise model** — Formalize security when up to $t-1$ shares are compromised *before* the ceremony begins. Conjectured to follow from Shamir privacy, but a complete game-based reduction is needed.
5. **Secure erasure as a formal primitive** — Model Destroy as a cryptographic oracle, define its security properties independently of the SMA model, and provide a composability proof. This remains the deepest open problem in the formal treatment.
6. **Formal treatment of Attested Key Existence (Definition 5.6)** — AKE is given as an instantiation of accountability / transparency-log notions (Küsters et al. 2010; CONIKS / CT) with an informal soundness argument. A game-based definition, a reduction to attestation-key unforgeability and hash-chain collision resistance, its composition with the EKS security properties, and **split-view / equivocation resistance** (gossip or multi-auditor cross-checking, which hash-chaining alone does not provide) remain to be written.

12. Conclusion

We have specified Ethereal Cryptography as an **architectural construction — not a new paradigm** — in which the complete signing key is assembled from threshold shares, used once, and erased within a formally bounded threat model, composing established techniques (forward secrecy, threshold sharing, key-insulated / intrusion-resilient models, secure erasure) with post-quantum signing. Its central notion, Quantum Forward Secrecy, is a **named configuration** of forward secrecy holding under an explicit, empirical epoch-lifetime invariant; it is **not** stronger than the threshold/MPC state of the art, which already achieves key non-existence (§10.3). The Ephemeral-Quantum Gap Theorem states this conditionally, and we mark the invariant as a deployment assumption throughout.

Composed with ML-DSA-87, the design *targets* Full Ethereal Security — no complete key existing at any time — but the current SPECTER build reaches this only approximately (§9.4): it momentarily materializes the key at signing and uses a dealer at genesis. The honest contribution is the specific engineered, audited composition, and a construction roadmap that closes the remaining gap by adopting standard-compatible threshold signing (FROST classically; threshold ML-DSA as it matures — Celi et al. 2026, Quorus; the frontier is young, and an early candidate, TALUS, was already broken (Niot 2026), which is exactly why the roadmap gates the post-quantum phase on external cryptanalysis before adoption). Theorem 6.1 now closes sequential composition for Case A and for Case B with the master secret hidden; the deepest open problems remaining are Destroy as a formal primitive (Open Problem 5) and the all-shares Case B sequential composition (Open Problem 1, narrowed by Theorem 6.1). SPECTER is a first deployed instantiation and a work in progress, not a finished result.

References

- Shamir, A. (1979). How to share a secret. *CACM*, 22(11).
- Shor, P. (1994). Algorithms for quantum computation. *FOCS*.
- Krawczyk, H. (2010). Cryptographic extraction and key derivation: The HKDF scheme. *CRYPTO*.
- Roetteler, M. et al. (2017). Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms. *ASIACRYPT*. IACR ePrint 2017/598.
- NIST (2024). FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA).
- NIST (2024). FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM).
- Bernstein, D.J. & Lange, T. (2017). Post-quantum cryptography. *Nature*, 549.
- Boneh, D. & Zhandry, M. (2013). Secure signatures and chosen ciphertext security in a quantum computing world. *CRYPTO*.
- Komlo, C. & Goldberg, I. (2020). FROST: Flexible Round-Optimized Schnorr Threshold Signatures. *SAC*.
- Connolly, D., Komlo, C., Goldberg, I. & Wood, C.A. (2024). RFC 9591: The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol. IRTF CFRG.
- Blakley, G.R. (1979). Safeguarding Cryptographic Keys. *AFIPS National Computer Conference*.
- Desmedt, Y. & Frankel, Y. (1989). Threshold Cryptosystems. *CRYPTO*.
- Diffie, W., van Oorschot, P. & Wiener, M. (1992). Authentication and Authenticated Key Exchanges. *Designs, Codes and Cryptography*.
- Herzberg, A., Jarecki, S., Krawczyk, H. & Yung, M. (1995). Proactive Secret Sharing. *CRYPTO*.
- Boneh, D. & Lipton, R. (1996). A Revocable Backup System. *USENIX Security*.
- Di Crescenzo, G., Ferguson, N., Impagliazzo, R. & Jakobsson, M. (1999). How to Forget a Secret. *STACS*.
- Dodis, Y., Katz, J., Xu, S. & Yung, M. (2002). Key-Insulated Public-Key Cryptosystems. *EUROCRYPT*.
- Itkis, G. & Reyzin, L. (2002). SiBIR: Signer-Base Intrusion-Resilient Signatures. *CRYPTO*.
- Perlman, R. (2005). The Ephemerizer: Making Data Disappear. *Sun Microsystems TR*.
- Geambasu, R., Kohno, T., Levy, A. & Levy, H. (2009). Vanish: Increasing Data Privacy with Self-Destructing Data. *USENIX Security*.
- Gennaro, R. & Goldfeder, S. (2020). One Round Threshold ECDSA (GG20). IACR ePrint 2020/540.
- Canetti, R., Gennaro, R., Goldfeder, S., Makriyannis, N. & Peled, U. (2021). UC Non-Interactive, Proactive, Threshold ECDSA (CGGMP21). *ACM CCS*.
- Signal (2023). The PQXDH Key Agreement Protocol.
- Celi, S., del Pino, R., Espitau, T., Niot, G. & Prest, T. (2026). Efficient Threshold ML-DSA. IACR ePrint 2026/013.
- Bienstock, A., de Castro, L., Escudero, D., Polychroniadou, A. & Takahashi, A. (2026). Quorus: Efficient, Scalable Threshold ML-DSA Signatures from MPC. *USENIX Security 2026*; IACR ePrint 2025/1163.
- Kao, L. & Chang, R. (2026). TALUS: FIPS-204-Exact Threshold ML-DSA via Boundary Clearance. arXiv:2603.22109v4 (23 Jul 2026). (*Cited only as a cautionary example. Broken by [Niot 2026]; the v4 revision rewrites the security analysis to address the attack, but that fix is not yet externally vetted — not a viable candidate as it stands.*)
- Niot, G. (2026). Key-Recovery Attacks on TALUS: A Cryptanalytic Note. IACR ePrint 2026/1386.
- Küsters, R., Truderung, T. & Vogt, A. (2010). Accountability: Definition and Relationship to Verifiability. *ACM CCS*; IACR ePrint 2010/236.

- Melara, M.S., Blankstein, A., Bonneau, J., Felten, E.W. & Freedman, M.J. (2015). CONIKS: Bringing Key Transparency to End Users. *USENIX Security*; IACR ePrint 2014/1004.